



검색증강생성

최근 수정 시각: 2026-08-09 19:25:35

분류: [IT 관련 용어](#)

목차

- 1. 개요
- 2. 배경
 - 2.1. LLM의 컷오프 문제
 - 2.2. 초기 해결 시도의 실패
 - 2.3. RAG의 등장
- 3. 작동 방식
 - 3.1. 전체 흐름
 - 3.2. 검색 단계의 핵심
 - 3.3. 생성 단계의 핵심
- 4. 장점
 - 4.1. 시간적 제약의 극복
 - 4.2. 부족한 지식 벌충
 - 4.3. 출처 투명성
- 5. 단점
 - 5.1. 지식과 능력의 간극
 - 5.2. 외국어 번역 품질 문제
 - 5.3. 외부 의존성
 - 5.4. 창의성 감퇴
- 6. 실제 적용 사례
 - 6.1. 검색 엔진 통합
 - 6.2. 전용 RAG 서비스
 - 6.3. 기업용 솔루션
 - 6.4. 프레임워크

1. 개요

검색증강생성, RAG(Retrieval-Augmented Generation)는 대규모 [언어 모델](#)(LLM)이 답변을 생성하기 전에 외부 지식 베이스에서 관련 정보를 검색하여 활용하도록 하는 기술이다. 쉽게 말하면, 모델이 모르는 것을 백과사전에서 찾

아본 후 자기 말로 설명하는 것과 유사하다.

RAG 기술은 Facebook AI Research(FAIR)에서 개발되었으며, 2020년 발표된 논문 「Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks」에서 처음 제안되었다. 이후 정보 검색, 지식 기반 질의응답, 문서 요약 등 다양한 자연어 처리 연구 분야에서 주목받고 있다.

2. 배경

2.1. LLM의 컷오프 문제

대규모 언어 모델의 가장 고질적인 문제는 **학습 종료 시점(cutoff) 이후의 정보를 알 수 없다**는 것이다. 일례로 1월에 학습이 끝난 모델이라면 2월만 되어도 그 정보는 알지 못한다. 학습된 정보 내의 추세로 예측은 할 수 있겠지만, 그걸 벗어난 정보가 필요한 상황이라면 오히려 오답만 유도하는 꼴이 되기 쉽다. 현대 사회에서 몇개월만 돼도 차이는 매우 크다. 선거 결과, 기업 실적, 기술 동향, 심지어 유행어까지 끊임없이 변한다. 사용자가 '최근' 정보를 물으면 모델은 낡은 답을 하거나, 아예 모른다고 인정하거나, 최악의 경우 **지어낸 정보를 제공한다**.

이 컷오프 문제는 LLM이 등장한 순간부터 지금까지 해결되지 않은 근본적 한계다. 사용자들은 "이 정보는 2024년 1월 기준입니다"라는 단서를 보면서 최신 정보를 얻지 못하는 것에 불만을 토로해왔다. 학습 데이터에 포함되지 않은 특수 분야 지식이나 기업 내부 문서 같은 정보 역시 LLM이 답할 수 없는 영역이었다.

2.2. 초기 해결 시도의 실패

컷오프 문제를 해결하기 위한 여러 시도가 있었으나, 각각 한계가 명확했다.

- **모델 재학습**

새 데이터로 모델을 재학습하는 것은 가장 직관적인 해결책이지만, 수백억 원의 비용이 든다. 매달, 심지어 매주 재학습할 수는 없다. 더구나 재학습 과정에서 기존에 잘 하던 작업의 성능이 저하되는 "catastrophic forgetting" 문제도 발생할 수 있다.

- **초기 검색 Tool의 무능**

초기에는 LLM에 검색 기능을 붙이는 시도가 있었다. 그러나 이 검색 Tool은 극히 초보적인 수준이었다. 사용자가 "야구의 역사와 현대 야구의 전략적 변화"에 대해 물으면, 모델은 단순히 "야구"라는 키워드로 검색하는 식이었다.^[1] 검색어 생성이 단순무식했고, 검색 결과를 답변에 통합하는 방식도 조잡했다. 사용자들은 이런 검색 기능을 불신하게 되었고, 차라리 검색을 쓰지 않고 모델의 멍청함을 조롱하는 쪽을 택했다. "검색 기능이 있으면 뭐하나, 제대로 검색도 못하는데"라는 반응이 지배적이었다.

- **수동 병용의 불편함**

사용자가 직접 검색엔진에서 정보를 찾고, 그것을 복사해서 LLM에게 정보로 제공한 다음 질의를 곁들이는 방식도 가능했다. 하지만 이것은 임시방편일 뿐이었다. 사용자가 검색과 질문을 별도로 수행해야 하므로 번거롭고, LLM의 장점인 "대화만으로 정보를 얻는다"는 편의성이 완전히 사라졌다.

2.3. RAG의 등장

Facebook AI Research는 2020년 논문에서 RAG를 제안했다. 핵심 아이디어는 **검색의 체계화**였다. 단순히 키워드를 검색엔진에 던지는 것이 아니라, 다음과 같은 과정을 자동화하고 통합했다.

1. 사용자 질문을 분석하여 적절한 검색어를 생성
2. 관련 문서를 검색하고 관련도에 따라 순위화
3. 검색된 정보를 컨텍스트로 활용하여 답변 생성

이 모든 과정을 통합적으로 자동화하여, 사용자는 그냥 질문하면 되고, 시스템이 알아서 필요하면 검색하고 답한다. 초기 검색 Tool의 단순무식함을 넘어서는 정교한 통합이었다. 이후 RAG는 [Perplexity](#), Microsoft Copilot, ChatGPT 웹 브라우징 기능 등 다양한 실제 서비스에 적용되며 LLM의 한계를 보완하는 핵심 기술로 자리잡았다.

3. 작동 방식

RAG의 작동 원리는 사람이 모르는 것을 백과사전에서 찾아본 후 자기 말로 설명하는 것과 유사하다. 사용자가 질문하면, 시스템은 먼저 관련 정보를 검색한 다음, 그 정보를 바탕으로 답변을 생성한다. 이 과정은 검색(Retrieval)과 생성(Generation)이라는 두 단계가 자동으로 연결되어 작동한다. 전통적인 LLM이 자신의 학습 데이터에만 의존하여 답변했다면, RAG는 필요할 때마다 외부 지식을 실시간으로 참조하여 더욱 정확하고 최신의 정보를 제공할 수 있다.

3.1. 전체 흐름

RAG 시스템은 4단계로 작동한다. 각 단계는 유기적으로 연결되어 있으며, 한 단계의 성공 여부가 다음 단계의 품질을 결정한다.

- ① **벡터화(Vectorization)**

사용자의 질문을 수치 벡터로 변환한다. 컴퓨터가 언어의 의미를 비교하려면 텍스트를 숫자로 표현해야 하기 때문이다. 예를 들어 "파리 여행 추천 명소"라는 질문은 [0.23, 0.87, -0.45, 0.12, ...] 같은 수백에서 수천 개의 숫자로 이루어진 벡터가 된다. 이 벡터는 단어의 의미를 수학적으로 표현한 것으로, 의미가 비슷한 문장들은 비슷한 벡터 값을 갖게 된다.

이 과정을 "임베딩(embedding)"이라고 부르며, 사전 학습된 임베딩 모델이 담당한다. 중요한 점은 단순히 단어의 출현 빈도가 아니라 의미적 관계를 포착한다는 것이다. "자동차"와 "차량"은 다른 단어지만 의미가 유사하므로 벡터 공간에서 가까운 위치에 놓인다. 마찬가지로 "파리 여행"과 "프랑스 수도 관광"도 겹치는 단어가 없지만 의미적으로 연관되어 있어 벡터상 가까운 거리에 위치하게 된다.

벡터화의 품질은 최종 결과에 결정적 영향을 미친다. 만약 임베딩 모델이 질문의 의도를 제대로 파악하지 못해 엉뚱한 방향의 벡터를 생성하면, 이후 검색 단계에서 아무리 좋은 문서 데이터베이스를 갖추고 있어도 관련 없는 정보만 검색하게 된다.

- ② **검색(Retrieval)**

벡터 데이터베이스에서 질문과 의미적으로 유사한 문서를 검색한다. 데이터베이스에 저장된 모든 문서 역시 미리 벡터로 변환되어 있으며, 질문 벡터와의 유사도를 계산하여 관련성 높은 문서들을 찾아낸다. 이 과정은 고차원

공간에서의 "거리 측정"으로 이루어진다. 벡터 간의 코사인 유사도(cosine similarity)나 유클리드 거리(Euclidean distance) 같은 수학적 척도를 사용하여, 질문 벡터와 가장 가까운 문서 벡터들을 선별한다.

예를 들어 "2024년 파리 올림픽 금메달 순위"라는 질문이 들어왔다고 가정하자. 시스템은 벡터 데이터베이스에서 "올림픽", "메달", "2024년" 같은 키워드가 포함된 문서뿐만 아니라, "스포츠 대회 성적", "국가별 순위" 같은 의미적으로 관련된 문서들도 함께 검색한다. 이것이 전통적인 키워드 기반 검색과의 핵심적 차이점이다.

벡터 데이터베이스는 일반적으로 FAISS, Pinecone, Weaviate 같은 전문 시스템을 사용한다. 이들은 수억 개의 문서 벡터 중에서도 밀리초 단위로 유사 문서를 찾아낼 수 있도록 최적화되어 있다. 데이터베이스에 저장되는 문서는 웹페이지, 논문, 기업 내부 문서, 뉴스 기사 등 다양할 수 있으며, 시스템 구축자가 어떤 지식 베이스를 구축하느냐에 따라 RAG의 활용 범위가 결정된다.

● ③ 순위화(Ranking)

검색된 문서들을 관련도순으로 정렬한다. 검색 단계에서는 보통 수십 개에서 수백 개의 문서가 후보로 추출될 수 있다. 하지만 모든 문서를 다 생성 단계로 전달하면 두 가지 문제가 발생한다. 첫째, LLM이 처리해야 할 정보량이 과도해져서 컨텍스트 윈도우 제약에 걸린다. 둘째, 관련성이 낮은 문서들이 섞여 있으면 오히려 생성 품질이 저하된다.

따라서 순위화 단계에서는 검색된 문서들 중 상위 3~10개 정도만 선별한다. 이때 단순히 벡터 유사도만 보는 것이 아니라, 문서의 최신성, 출처의 신뢰도, 문서 내에서 관련 정보가 차지하는 비중 등을 종합적으로 고려할 수 있다. 일부 고급 RAG 시스템에서는 이 단계에서 "재순위화(ReRank)" 모델을 추가로 사용하기도 한다. 재순위 모델은 질문과 각 문서를 함께 입력받아, 단순한 벡터 유사도를 넘어서 실제 관련성을 더 정교하게 평가한다.

예를 들어 "파리 여행 추천 명소"라는 질문에 대해 벡터 유사도만으로는 "파리의 역사"라는 문서와 "파리 에펠탑 관광 정보"라는 문서가 비슷한 점수를 받을 수 있다. 하지만 재순위 모델은 사용자가 "추천 명소"를 원한다는 의도를 파악하여 후자에 더 높은 순위를 부여한다.

● ④ 생성(Generation)

대규모 언어 모델이 검색된 정보를 컨텍스트로 활용하여 답변을 생성한다. 이 단계는 RAG의 최종 산출물이 만들어지는 곳으로, 모델의 실제 언어 능력이 시험받는 구간이다. LLM은 사용자의 원래 질문과 검색된 문서들을 함께 입력받는다.

이때 프롬프트는 대략 다음과 같은 형태를 띈다.

다음은 사용자 질문과 관련된 참고 문서들입니다:

[문서 1 내용]

[문서 2 내용]

[문서 3 내용]

사용자 질문: 2024년 파리 올림픽 금메달 순위는?

위 참고 문서의 내용을 바탕으로 질문에 답변하세요.

모델은 이 정보를 종합하여 사용자의 질문에 맞는 형태로 답변을 재구성한다. 중요한 점은 단순히 검색 결과를 복사-붙여넣기하는 것이 아니라는 것이다. 모델은 여러 문서에 흩어진 정보를 통합하고, 불필요한 부분은 제거하며, 질문의 의도에 정확히 부합하는 형태로 가공해야 한다.

만약 검색된 문서에 "미국은 올림픽에서 총 126개의 메달을 획득했으며, 이 중 금메달은 40개였다"라고 적혀 있다면, 모델은 "2024년 파리 올림픽에서 미국이 금메달 40개로 1위를 차지했습니다"처럼 질문에 직접 답하는 형태로 재구성한다. 또한 여러 문서의 정보가 상충될 경우, 출처의 신뢰도나 최신성을 고려하여 어느 정보를 우선할지 판단해야 한다.

일부 RAG 시스템은 생성 단계에서 답변과 함께 참고한 문서의 출처를 함께 제시하기도 한다. 이는 사용자가 정보의 신뢰성을 직접 검증할 수 있게 하여 투명성을 높인다.

3.2. 검색 단계의 핵심

검색 단계의 가장 큰 문제는 **키워드 매칭만으로는 의미적 관련성을 포착할 수 없다**는 점이다. 전통적인 검색 엔진은 질문에 포함된 단어가 문서에 있는지를 찾는 방식이었다. 예를 들어 "자동차가 고장났어요"라는 질문에는 "자동차" 또는 "고장"이라는 단어가 포함된 문서만 검색했다. 이 방식의 문제점은 "차량 수리 방법"이라는 문서를 찾지 못한다는 것이다. "자동차"와 "차량"이 같은 의미임을 이해하지 못했기 때문이다.

더 심각한 문제는 동음이의어나 다의어를 구분하지 못한다는 점이다. "사과"라는 단어로 검색하면 과일 사과와 용서를 구하는 행위인 사과가 섞여서 나온다. "배"로 검색하면 과일, 신체 부위, 교통수단이 모두 검색된다. 맥락을 이해하지 못하는 키워드 검색의 근본적 한계였다.

RAG는 이를 **벡터 임베딩(vector embedding)**으로 해결한다. 텍스트를 고차원 공간의 점으로 표현하여, 의미가 비슷한 텍스트들은 공간상에서 가까운 위치에 놓이게 만든다. 이는 수백 차원에서 수천 차원의 벡터 공간에서 이루어지며, 각 차원은 언어의 특정 의미적 특성을 나타낸다. 예를 들어 어떤 차원은 "교통수단"의 의미를, 다른 차원은 "과일"의 의미를 포착할 수 있다.

이렇게 하면 표현이 다르더라도 의미가 같으면 관련 있다고 판단할 수 있다. "파리 여행"과 "프랑스 수도 관광"은 겹치는 단어가 없지만, 의미적으로 연결되어 벡터 공간에서 가까운 위치에 놓인다. "자동차 고장"과 "차량 수리"도 마찬가지다. 심지어 언어가 달라도 의미가 같으면 가까운 벡터를 가질 수 있다. 다국어 임베딩 모델을 사용하면 "car"와 "자동차"가 유사한 벡터로 표현된다.

다만 이 과정의 품질은 **임베딩 모델의 성능**에 크게 달려있다. 모델이 언어의 의미를 얼마나 정확하게 수치화할 수 있는가에 따라 검색 정확도가 결정된다. 특히 전문 분야나 비영어권 언어에서는 임베딩 품질이 떨어지는 경우가 많다. 일반적인 영어 텍스트로 학습된 임베딩 모델은 의학 용어나 법률 용어의 미묘한 차이를 잘 포착하지 못할 수 있다. 한국어 같은 비영어권 언어는 학습 데이터 자체가 부족하여 임베딩 품질이 영어보다 낮은 경우가 흔하다.

또한 검색 대상이 되는 문서들을 어떤 크기로 분할하는가(청킹, chunking) 역시 결과에 영향을 준다. 문서를 너무 작은 단위로 쪼개면 문맥 정보가 손실되어 의미 파악이 어려워진다. 반대로 너무 큰 단위로 유지하면 관련 없는 정보가 많이 섞여서 생성 단계의 부담이 커진다. 대부분의 RAG 시스템은 200~500 토큰(대략 150~400 단어) 정도의 청크를 사용하지만, 이는 문서의 특성과 사용 목적에 따라 달라진다.

3.3. 생성 단계의 핵심

생성 단계의 핵심 과제는 **검색 결과를 단순히 복사하는 것이 아니라, 질문에 맞게 재구성해야 한다**는 점이다. 검색된 문서는 질문과 관련은 있지만, 사용자가 원하는 형태로 작성되어 있지는 않다. 예를 들어 "파리의 인구는 얼마인가?"라는 질문에 대해 검색된 문서에는 "파리 광역권은 약 1,200만 명이 거주하는 프랑스 최대 도시권이며, 파리시

자체의 인구는 약 220만 명이다"라고 적혀 있을 수 있다.

이 경우 모델은 여러 판단을 해야 한다. 첫째, 사용자가 묻는 "파리"가 파리시를 의미하는가, 파리 광역권을 의미하는가? 문맥상 대부분의 사용자는 파리시를 묻지만, 일부는 광역권을 의미할 수 있다. 둘째, 두 숫자를 모두 제시할 것인가, 하나만 제시할 것인가? 셋째, "약"이라는 표현을 유지할 것인가, "약 220만 명" vs "220만 명"의 차이를 어떻게 다룰 것인가?

이런 판단은 단순한 정보 추출이 아니라 **이해와 분석**을 요구한다. 생성 모델은 검색 결과의 의미를 파악하고, 질문의 의도를 해석하며, 두 가지를 조합하여 적절한 답변을 구성해야 한다.

더 복잡한 상황은 검색 결과가 여러 개이고 서로 상충되는 정보를 담고 있을 때다. "커피가 건강에 좋은가?"라는 질문에 대해 한 문서는 "커피는 항산화 물질이 풍부하여 심혈관 질환 예방에 도움이 된다"고 하고, 다른 문서는 "과도한 카페인 섭취는 불안감과 수면 장애를 유발할 수 있다"고 말할 수 있다. 모델은 이 두 정보가 모순되는 것이 아니라 서로 다른 측면을 다루고 있음을 이해하고, "적정량의 커피는 건강에 이로운 측면이 있지만, 과다 섭취는 부작용을 일으킬 수 있다"는 식으로 균형잡힌 답변을 생성해야 한다.

이 과정에서 **모델의 기본 능력**이 결정적이다. 모델이 검색 결과의 의미를 올바르게 파악하지 못하면, 정확한 정보를 가져와도 틀린 답변을 생성할 수 있다. 예를 들어 2025년 출시된 **GPT-5**는 UNCURK의 문제점을 찾아보라 할 때 이승만과 박정희 정권을 추인했다는 사실을 응답에 빼먹고 엉뚱한 **냉전** 구도만 이야기하는 편이다. 이는 RAG가 단순히 검색 기능만 추가한다고 해결되는 것이 아니라, **모델 자체의 이해력과 판단력이 뒷받침되어야 함**을 보여준다.

또한 **검색 결과의 품질이 최종 답변의 품질을 좌우**한다는 근본적 한계가 있다. 아무리 생성 모델이 뛰어나더라도, 검색 단계에서 잘못된 정보나 관련 없는 정보를 가져오면 올바른 답변을 생성할 수 없다. **쓰레기가 들어가면 쓰레기가 나온다**는 컴퓨팅의 오랜 격언이 RAG에도 그대로 적용된다. 이것이 RAG가 "외부 의존성"이라는 태생적 약점을 갖는 이유다. 검색 데이터베이스의 품질, 검색 알고리즘의 정확성, 그리고 원본 문서의 신뢰성이 모두 최종 결과에 영향을 준다.

마지막으로, 생성 단계에서는 **환각(hallucination) 문제**가 여전히 존재할 수 있다. RAG가 환각을 완전히 제거하지는 못한다. 모델이 검색 결과를 무시하고 자신의 학습 데이터나 잘못된 추론에 기반하여 정보를 지어낼 수 있기 때문이다. 특히 검색 결과가 불완전하거나 애매할 때, 모델은 그 빈틈을 자신의 "상상"으로 채우려는 경향이 있다. 이를 방지하려면 생성 프롬프트에서 "반드시 제공된 문서의 정보만을 사용하여 답변하고, 확실하지 않으면 모른다고 말하라"는 지시를 명확히 해야 한다. 하지만 이것도 완벽한 해결책은 아니며, 모델의 근본적인 언어 이해 능력과 지시 준수 능력에 달려있다.

4. 장점

4.1. 시간적 제약의 극복

RAG의 가장 명확한 장점은 **모델의 학습 cutoff 이후의 정보에 접근할 수 있다**는 것이다. 2025년 1월에 학습이 끝난 모델이라도, RAG를 통해 10월의 최신 뉴스나 데이터를 검색하여 답변할 수 있다. 이는 RAG 없이는 불가능한 기능이다.

이 장점은 시간에 민감한 정보를 다룰 때 특히 중요하다. 주식 시장 동향, 최신 연구 결과, 정치 상황, 날씨 정보 등은 매일 또는 매시간 변하므로, 모델의 학습 데이터만으로는 답변할 수 없다. RAG는 이런 정보를 실시간으로 검색하여

제공함으로써 LLM의 실용성을 크게 높인다.

예를 들어 "오늘 서울 날씨는?"이라는 질문에 학습 데이터만 있는 모델은 "2025년 1월 이후의 정보는 알 수 없습니다"라고 답할 수밖에 없다. 반면 RAG를 사용하는 시스템은 기상청 웹사이트를 검색하여 "오늘 서울은 맑고 최고기온 15도입니다"라고 정확하게 답변할 수 있다.

다만 **검색어를 제대로 생성할 능력**은 여전히 필요하다. 모델이 "오늘 서울 날씨"를 "서울 기후 특성"으로 잘못 이해하면, 일반적인 기후 정보만 검색하여 오늘의 날씨를 알려주지 못한다. 또한 LLM은 별도의 Tool이나 시스템 프롬프트 없이는 오늘 날씨가 알지 못하는데, 따라서 **오늘 날씨가 오인**하여 미래나 과거의 날씨를 검색할 가능성도 충분하다. 따라서 이 장점도 모델의 기본 이해력에 어느 정도 달려있지만, 다른 장점들에 비하면 가장 확실한 편이다.

4.2. 부족한 지식 벌충

RAG는 **모델이 학습하지 못한 특정 정보를 검색으로 보완**할 수 있다. 이는 학습 데이터에 포함되지 않은 전문 분야 지식, 기업 내부 문서, 특정 커뮤니티의 정보 등을 활용할 수 있게 한다.

예를 들어 일본 애니메이션에 대한 정보가 부족한 모델이라도, RAG를 통해 애니메이션 데이터베이스를 검색하여 "이 작품의 감독은 누구인가?", "이 캐릭터의 성우는?" 같은 질문에 답할 수 있다. 마찬가지로 특정 기업의 내부 정책이나 제품 사양서 같은 정보도, 해당 문서들을 벡터 데이터베이스에 저장해두면 RAG를 통해 접근할 수 있다.

이것의 핵심은 **선택적 보완**이다. 모델이 잘 아는 일반적인 질문에는 학습 데이터만으로 답하고, 모르는 특수한 질문에는 검색을 통해 보완한다. 이론적으로는 매우 효율적인 방식이다.

문제는 **모델이 "자기가 모른다는 것을 알 정도로" 똑똑해야 한다**는 점이다. 멍청한 모델은 모르는데도 아는 척하거나 (환각), 검색해야 할 타이밍을 판단하지 못한다. 또한 검색된 정보를 제대로 이해하지 못하면, 좋은 자료를 가져와도 활용하지 못하거나 잘못 해석한다. 따라서 이 장점은 모델의 메타인지 능력과 이해력에 크게 달려있다.

4.3. 출처 투명성

RAG 시스템은 답변과 함께 **참고한 문서의 출처를 제시**할 수 있다. 이는 사용자가 정보의 근거를 직접 확인할 수 있게 하여 투명성을 높인다. [Perplexity](#) 같은 서비스는 각 문장마다 각주 형식으로 출처를 명시하며, Microsoft Copilot도 답변 하단에 참고 링크를 제공한다.

출처 제공의 진짜 가치는 **검증 가능성**이다. 사용자는 "이 정보가 정말 맞는가?"를 궁금해할 때 출처 링크를 클릭하여 원본을 확인할 수 있다. 또한 더 자세한 정보가 필요하면 원본 문서를 직접 읽을 수 있다. 이는 전통적인 LLM이 제공할 수 없었던 기능이다.

다만 출처가 있다고 내용이 맞는 것은 아니다. RAG 시스템이 신뢰할 수 없는 웹사이트를 검색할 수도 있고, 검색된 정보를 잘못 해석하여 출처와 다른 내용을 답변할 수도 있다. 심지어 출처가 명시되어 있으면 사용자가 더 쉽게 믿는 경향이 있어, 잘못된 정보가 더 위험할 수도 있다.

따라서 출처 투명성은 신뢰성의 증거가 아니라 **검증 가능성의 제공**이다. "이 답변을 믿어도 된다"는 보증이 아니라, "직접 확인해 볼 수 있다"는 수단을 주는 것이다. 검증 책임은 여전히 사용자에게 있다.

5. 단점

5.1. 지식과 능력의 간극

RAG의 가장 근본적인 한계는 **검색으로 정보는 가져올 수 있지만, 그것을 이해하고 활용하는 능력까지 보충하지는 못한다**는 점이다. 이는 2025년 출시된 GPT-5가 극명하게 보여준 문제다.

- **검색어 생성 실패**

모델의 이해력이 부족하면 적절한 검색어를 생성하지 못한다. "최근 한국 영화의 경향"을 물었는데 단순히 "한국 영화"로만 검색하면, 역사적 정보만 나오고 최근 경향은 찾지 못한다. 더 심각한 경우, 질문의 핵심을 완전히 오해하여 엉뚱한 키워드로 검색하기도 한다.

- **검색 결과 오해석**

올바른 정보를 검색해도 모델이 그것을 제대로 이해하지 못하는 경우가 있다. 분석력 부족으로 인해 정확한 정보를 노이즈로 치부하거나, 틀린 맥락으로 재해석하여 인용한다.

- **확증편향적 검색**

추론력이 부족한 모델은 환각 낀 결론을 먼저 정하고, 그것을 "증명"하기 위해 검색하는 경우가 있다. 결론이 틀렸음에도 그것을 뒷받침하는 정보만 선별적으로 검색하거나, 검색 결과를 왜곡 해석하여 자신의 환각을 정당화한다. 이는 검색이 오히려 환각을 강화하는 역효과를 낳는다.

- **벤치마크와 실사용의 괴리**

이런 방식은 벤치마크 점수는 유지하면서 실제 사용 경험은 크게 저하시킨다. 벤치마크는 정답이 명확한 문제 중심이라 검색으로 보완 가능하지만, 실제 대화나 창작, 복잡한 맥락 이해는 모델의 내재적 능력에 크게 달려있기 때문이다. 일부 제공사는 비용 절감을 위해 모델 품질을 낮추고 RAG로 때우려 하지만, 이는 결국 실패한다. **RAG는 능력 부족을 절대 가릴 수 없다.**

5.2. 외국어 번역 품질 문제

RAG는 다국어 환경에서 구조적 약점을 보인다. 특히 영어 중심으로 학습된 모델이 비영어권 쿼리를 처리할 때 심각한 품질 저하가 발생한다.

문제의 메커니즘은 다음과 같다. 영어 중심 모델이 한국어 질문을 받으면, 질문을 영어로 번역하거나 영어 키워드로 검색한다. 그 결과 영어 문서를 검색하게 되고, 이를 다시 한국어로 번역하여 제시한다. 이 과정에서 맥락과 뉘앙스가 완전히 파괴되고 한국어 문법이 제대로 반영되지도 않은^[2] 괴상한 내용이 작성된다.

GPT-5 추론 모델에서 큰 문제가 되는 부분이 바로 이것이다. 한국어 질문에 대해 영어 문서를 검색한 후, 어색한 한국어로 번역하거나 아예 영어 문장의 구조 그대로 때려박아 답변한다. 그런 식의 대답은 **불쾌한 골짜기**와 유사한 효과를 일으킨다. 사용자가 답변의 이해를 거부하고 해당 모델을 욕하는 것이다.

결과적으로 비영어권 사용자는 **부정확한 번역, 어색한 표현, 문화적 맥락 무시**라는 삼중고를 겪는다. RAG가 다국어를 지원한다고 주장하지만, 실제로는 영어 중심 설계의 한계를 넘지 못한다.

5.3. 외부 의존성

RAG는 **외부 지식에 의존하므로, 검색된 정보가 올바르지 않으면 부정확한 결과가 나온다.** 이는 피할 수 없는 구조적 한계다.

검색 대상이 되는 데이터베이스의 품질이 최종 답변을 결정한다. 잘못된 정보, 편향된 정보, 낡은 정보가 데이터베이스에 있으면, 아무리 모델이 뛰어나도 틀린 답변을 생성한다. 인터넷은 정보의 바다인 동시에 정보의 쓰레기통이기 때문에, 웹 검색 기반 RAG는 항상 이 위험에 노출된다.

또한 검색 시스템 자체의 장애나 한계도 문제가 된다. 검색 API가 다운되면 RAG는 작동하지 않는다. 검색 속도가 느리면 답변 생성 시간이 길어져 사용자 경험이 나빠진다. 특정 사이트가 robots.txt로 크롤링을 차단하면 그 정보에는 접근할 수 없다.

이는 LLM만 사용할 때는 없던 새로운 종류의 실패 가능성을 추가한다. 전통적인 LLM은 느려도 항상 답변하지만, RAG는 검색이 실패하면 답변 자체를 생성하지 못할 수 있다.

5.4. 창의성 감퇴

RAG의 검색 기반 응답에 과도하게 의존하면 모델의 독창적 생성 능력이 저하될 수 있다. 이는 특히 창작이나 추상적 사고가 필요한 작업에서 문제가 된다.

검색은 본질적으로 기존에 존재하는 정보를 찾는 과정이다. 따라서 RAG는 **사실 확인**에는 강하지만, **새로운 아이디어 생성**에는 약하다. 소설 쓰기, 시 창작, 가상 시나리오 상상 같은 작업은 검색할 자료가 없으므로, RAG가 도움이 되지 않는다.

더 심각한 것은 GPT-5에서 보이듯 RAG에 과도하게 의존하도록 설계된 모델은 검색 없이는 제대로 작동하지 못하는 경향을 보인다는 것이다. 창의적 질문에도 무리하게 검색을 시도하거나, 검색어 및 검색 결과를 짜맞추려다 오히려 이상한 답변을 생성하거나, 아예 검색하지 말라는 프롬프트를 거부하기도 한다.

또한 검색 결과에 얽매어 **예상 밖의 관점이나 독특한 해석**을 제시하지 못하는 경향도 있다. 검색된 문서들이 제시하는 주류 의견만 반복하게 되어, 모델 자체의 종합적 사고나 참신한 접근이 사라진다. 희귀한 사례를 설명하고 이에 대한 대책을 요구할 때도 통상적인 사례만 앵무새처럼 읊으며 제대로 응답하지 못하는 경향이 있다.^[3] 이는 RAG가 잘못 사용되었을 때의 부작용이지만, 현실에서 자주 관찰되는 문제다.

6. 실제 적용 사례

6.1. 검색 엔진 통합

- **Microsoft Copilot (구 Bing Chat)**

2023년 2월 출시된 Microsoft Copilot은 RAG의 가장 대표적인 상용 사례다. GPT-4 모델과 Bing 검색 엔진을 통합하여, 사용자의 질문에 대해 웹을 실시간으로 검색한 후 그 결과를 바탕으로 답변을 생성한다. 답변에는 각주

형식으로 출처 링크가 제공되어, 사용자가 정보의 근거를 직접 확인할 수 있다. 이미지 검색 결과도 활용 가능하며, 최신 뉴스나 날씨 같은 실시간 정보를 제공하는 데 강점을 보인다.

- **Google AI Overviews (구 Search Generative Experience)**

2023년 5월 발표된 Google의 AI Overviews는 검색 결과 상단에 AI가 생성한 요약を提供한다. PaLM 2 모델과 Google의 방대한 검색 인덱스를 활용하여, 사용자가 여러 웹페이지를 클릭하지 않아도 핵심 정보를 한눈에 볼 수 있게 한다. 2024년 "AI Overviews"로 이름을 변경하며 전면 확대되었고, 현재 수많은 검색 쿼리에 대해 AI 요약を提供한다.

6.2. 전용 RAG 서비스

- **Perplexity**

Perplexity는 RAG를 핵심으로 설계된 질의응답 서비스다. 사용자의 질문에 대해 웹 검색을 통해 관련 정보를 수집하고, 이를 바탕으로 답변을 생성한다. 가장 큰 특징은 **철저한 출처 명시**다. 답변의 각 문장마다 각주 형식으로 참고한 웹페이지 링크가 제공되며, 검색 과정도 사용자에게 투명하게 공개된다. Pro 버전은 더 깊이 있는 검색과 분석을 제공하며, Perplexity Academic 기능은 학술 논문 검색에 특화되어 있다.

- **ChatGPT의 웹 브라우징 기능**

OpenAI의 ChatGPT는 2023년 5월부터 "Browse with Bing" 기능을 베타로 제공했다. 여러 차례 중단과 재개를 거쳐 현재는 Plus 이상 요금제에서 사용 가능하다. 사용자가 최신 정보를 요구하면 자동으로 웹 검색을 실행하여 관련 정보를 수집하고, 검색한 웹페이지 URL을 답변에 포함한다. 2021년 연구 프로젝트였던 WebGPT의 기술이 실제 서비스로 구현된 것이다.

- **Gemini Notebook**

Google의 Gemini 계열 **대규모 언어 모델(LLM)**을 기반으로 사용자가 제공한 문서 내용을 실시간으로 참고하여 응답을 생성한다.

6.3. 기업용 솔루션

- **AWS Kendra + Bedrock**

Amazon Web Services는 Kendra(지능형 검색 서비스)와 Bedrock(LLM 서비스)를 결합하여 기업용 RAG 솔루션을 제공한다. 기업 내부 문서, 데이터베이스, 파일 시스템을 검색 대상으로 삼아, 직원들이 자연어로 질문하면 관련 내부 정보를 검색하여 답변한다. 접근 권한 관리가 통합되어 있어, 각 직원은 자신이 열람 가능한 문서만 검색 대상으로 삼는다.

- **Azure Cognitive Search + OpenAI**

Microsoft는 Azure Cognitive Search와 Azure OpenAI Service를 통합하여 엔터프라이즈 RAG 스택을 제공한다. 기업들은 자사의 문서를 Azure에 업로드하고 인덱싱한 후, GPT 모델과 연결하여 사내 지식 베이스 챗봇을 구축할 수 있다. 보안과 컴플라이언스 기능이 강화되어 있어, 금융이나 의료 같은 규제 산업에서도 사용 가능하다.

6.4. 프레임워크

- LangChain과 LlamaIndex는 RAG 시스템을 구축하기 위한 오픈소스 프레임워크다. 개발자들은 이 도구를 사용하여 자체 RAG 애플리케이션을 만들 수 있다. 다양한 벡터 데이터베이스, 임베딩 모델, LLM을 선택하여 조합할 수 있어 유연성이 높다. 스타트업부터 대기업까지 수많은 조직이 이 프레임워크를 채택하여 맞춤형 RAG 시스템을 운영 중이다.
- LangChain 생태계에는 LangGraph, LangSmith도 있다. 최근에는 [Computer Use](#), [AI 에이전트](#) 등의 동적 워크플로를 RAG에 접목시키기 위해 DeepAgents도 나왔다. LangChain에 [Playwright](#), Browser Use을 통합할 수도 있다.



이 저작물은 [CC BY-NC-SA 2.0 KR](#)에 따라 이용할 수 있습니다. (단, 라이선스가 명시된 일부 문서 및 삽화 제외)
기여하신 문서의 저작권은 각 기여자에게 있으며, 각 기여자는 기여하신 부분의 저작권을 갖습니다.

나무위키는 백과사전이 아니며 검증되지 않았거나, 편향적이거나, 잘못된 서술이 있을 수 있습니다.

나무위키는 위키위키입니다. 여러분이 직접 문서를 고칠 수 있으며, 다른 사람의 의견을 원할 경우 직접 토론을 발제할 수 있습니다.